

Jashesh Pandya

25ME10097

CUDA Task

Parallel computing is a type of computation in which many calculations or processes are carried out simultaneously. Large problems can often be divided into smaller ones, which can then be solved simultaneously. As power consumption (and consequently heat generation) by computers has become a concern in recent years, parallel computing has become the dominant paradigm in computer architecture, mainly in the form of multi-core processors.

Bit-level Parallelism:

Bit-level parallelism is a form of parallel computing that increases the processor word size. Increasing the word size reduces the number of instructions the processor must execute in order to perform an operation on variables whose sizes are greater than the length of the word.

Instruction-level Parallelism:

These instructions can be reordered and grouped, then executed in parallel without changing the program's result. This is known as instruction-level parallelism.

ILP allows the compiler and the processor to overlap the execution of multiple instructions or even to change the order in which instructions are executed.

Consider the following program:

```
e = a + b
f = c + d
m = e * f
```

Operation 3 depends on the results of operations 1 and 2, so it cannot be calculated until both of them are completed.

However, operations 1 and 2 do not depend on any other operation, so they can be calculated simultaneously.

If we assume that each operation can be completed in one unit of time, then these three instructions can be completed in a total of two units of time, giving an ILP of 3/2.

Data Parallelism:

Data Parallelism is parallelization across parallel computing environments. One way to look at it is to divide an array of n elements into multiple equal parts and assign each part to a different processor.

If a task takes T time, splitting it across N computing environments makes it take $T/N + \text{Merge Overhead Time}$, where Merge Overhead Time is the time it takes for thread(s) to merge the results from different computing environments.

Merge Overhead Time effectively puts up a ceiling on how fast a process can be completed.

Task Parallelism:

Task Parallelism involves using multiple processors to run different tasks on the same data.

In a multiprocessor system, task parallelism is achieved when each processor executes a different thread (or process) on the same or different data

Parallel Computing:

Traditionally, [computer software](#) has been written for serial computation. To solve a problem, an algorithm is constructed and implemented as a serial stream of instructions. These instructions are executed by a CPU on a single computer. Only one instruction may execute at a time—after that instruction is finished, the next one is executed.

Parallel computing, on the other hand, uses multiple processing elements simultaneously to solve a problem. This is accomplished by breaking the problem into independent parts so that each processing element can execute its part of the algorithm simultaneously with the others. The processing elements can be diverse and include a single computer with multiple processors, several networked computers, specialized hardware, or any combination of the above.

Amdahl's Law indicates that optimal performance improvement is achieved by balancing enhancements to both parallelizable and non-parallelizable components of a task. Furthermore, it reveals that increasing the number of processors yields diminishing returns, with negligible speedup gains beyond a certain point.

Disadvantages of Parallel Computing:

Parallel computing can incur significant overhead in practice, primarily due to the cost of merging data from multiple processes. Specifically, inter-process communication and synchronization can lead to overheads that are substantially higher—often by two or more orders of magnitude—compared to processing the same data on a single thread. Therefore, the overall improvement should be carefully evaluated.

CPU v/s GPU (Hardware-wise)

Simply speaking The CPU has fewer cores (1-16), but each performs very well.

A GPU has lots of cores (ranging from 100 to 1000), but each of them individually doesn't perform as powerfully as a CPU does.

The difference is that CPUs are very capable of handling a wide range of tasks at a good speed. General-purpose computing tasks, complex calculations, and operations that are not very parallelizable are performed very well by a CPU.

GPUs, on the other hand, can handle repetitive calculations at a very large scale and can break down big processes into parallel chunks in order to handle them all in one go. This is why GPUs are really good at handling large amounts of data and images (which are made up of millions of pixels).

GPUs are specialized for highly parallel computations and devote more transistors to data processing units, while CPUs dedicate more transistors to data caching and flow control.

CUDA Parallel Programming Paradigm:

Question: 1. What is CUDA?

CUDA→ Compute Unified Device Architecture

CUDA (Compute Unified Device Architecture) is a proprietary parallel computing platform and application programming interface (API) that allows software to use certain types of graphics processing units (GPUs) for accelerated general-purpose processing, significantly broadening their utility in scientific and high-performance computing.

2. Why CUDA?

CUDA is both a software layer that manages data, giving direct access to the GPU and CPU as necessary, and a library of APIs that enable parallel computation for various needs. In addition to drivers and runtime kernels, the CUDA platform includes compilers, libraries, and developer tools to help programmers accelerate their applications.

3. How does the CUDA Parallel Programming Paradigm work?

The CUDA platform is accessible to software developers through CUDA-accelerated libraries, compiler directives such as OpenACC, and extensions to industry-standard programming languages including C, C++, Fortran and Python. C/C++ programmers can use 'CUDA C/C++', compiled to PTX with nvcc (Nvidia's LLVM-based C/C++ compiler) or by clang itself. Fortran programmers can use 'CUDA Fortran', compiled with the PGI CUDA Fortran compiler from The Portland Group. Python programmers can use the cuPyNumeric library to accelerate applications on Nvidia GPUs.

Compute Kernels, Thread Hierarchy, Memory Hierarchy

Compute Kernels

A kernel is a function executed on the GPU. A function that is executed once in a program is executed N times (value of N depends on the use) in parallel on a GPU.

In a C program, a kernel is defined as a **global** void function.

This global declaration specifier indicates that the function is intended for the GPU.

Thread Hierarchy

Simply speaking, Thread Hierarchy looks like this:

Threads → Thread Blocks → Grids

Thread Blocks → Clusters → Grid with Clusters

What it does is allow thread blocks to be co-scheduled on the GPU Processing Cluster (GPC) in the GPU, just as threads are co-scheduled within thread blocks on the same streaming multiprocessor(sm).

Threads can be identified using a one-, two-, or three-dimensional thread index, forming a one-, two-, or three-dimensional block of threads, called a thread **block**. This provides a natural way to compute across the elements of a domain, such as a vector, matrix, or volume.

Blocks are organized into a one-dimensional, two-dimensional, or three-dimensional grid of thread blocks.

The number of threads per block and the number of blocks per grid are specified in the `<<<...>>>` Syntax can be of type `int` or `dim3`.

Memory Hierarchy Local memory of threads and registers → Shared memory per thread block → Shared memory of all thread blocks in a cluster form Distributed Shared Memory → Global Memory Shared between all the kernels (Present in a grid with clusters).

Two more types of thread memory are accessible to all threads: Constant and Texture Memory Spaces.

The CUDA programming model also assumes that both the host and the device maintain their own separate memory spaces in DRAM, referred to as *host memory* and *device memory*, respectively. Therefore, a program manages the global, constant, and texture memory spaces visible to kernels through calls to the CUDA runtime. This includes device memory allocation and deallocation as well as data transfer between host and device memory.

Unified Memory provides *managed memory* to bridge the host and device memory spaces. Managed memory is accessible from all CPUs and GPUs in the system as a single, coherent memory image with a common address space. This capability enables oversubscription of

device memory and greatly simplifies porting applications by eliminating the need to explicitly mirror data between the host and device.

CUDA Functions

CUDA is not a language; it is a platform that enables communication with GPUs.

Here are some of the functions used in CUDA on C:

cudaMalloc:

Behaving similarly to malloc, cudaMalloc allocates space to a pointer. The difference here is that this space is allocated on the GPU.

Its syntax is: cudaMalloc(&p, size), where p is the pointer and size is the amount of space to allocate.

cudaMemcpy:

cudaMemcpy allows us to copy memory to and from GPUs. This transfer could be between various devices and spaces.

Its syntax is: cudaMemcpy (a, A, size, cudaMemcpy_____)

where a is the destination space for the data. A is the space where data is being copied from. Size is the size of the data, and the blank following cudaMemcpy specifies the direction of the transfer, i.e., the space/device from which and to which the data is being transferred

cudaFree:

cudaFree is used to free the GPU memory allocated by a CUDA function.

__syncthreads:

This function is used to synchronize thread operations. If the operation in one thread depends on the results of another __syncthreads is used in order to put these threads on the same page.

cudaDeviceSynchronize:

This function is used to synchronize the host (CPU) and the device (GPU). After a kernel is initiated, since the processing is taking place on the GPU, the CPU doesn't wait for its output and continues executing the rest of the program. In order to stop that from happening, we use cudaDeviceSynchronize to make sure things proceed in desired order.

Interesting thing to note?

Hello World!!

Using the clock function from the time library returned 0.00000 seconds for C and 0.113 seconds for CUDA. What is interesting is that the CUDA program as a whole took more time to process than the C program.

This made something clear: for tasks like these, the time spent in the CUDA overhead accounts for most of the computation time, whereas the CPU just blitzes the calculations.

Important thing to note: CUDA only deals with flat 1D arrays.

Matrix Addition

CPU → 0.00000 seconds

GPU → 0.01200 seconds

The argument that a 32x32 matrix is too small to make a difference is baffling to me, but fine.

Literally the same results came through with a 64x64 matrix. Very interesting indeed.